

output from the signature, we can see that this signature is actually updated.

As a unit we should be able to pass both the sleepy time unit, which is an enum, or one of the following with literal seconds, minutes, hours, days or weeks. For some reason, IDE does not take this and we know why.

In this particular case the code is so complicated that it is quite tough to understand which signature should be deduced from it without running it, because it is then almost equivalent to actually running the code. This is a very difficult problem, and we cannot think of annotations that will simplify the work for our IDE.

This is why stub files are proposed.

What are stub files?

Stub files are special files containing PEP-84. The main purpose of these files is to hold the typing information for modules. They are just Python files with the extension 'pyi'. If an IDE finds a corresponding stub file for a module, it will analyse the stub file instead of the original file for syntax highlighting.

What to add to stub files

- Public members definitions
- Annotations
- Docstrings
- Final function signatures without decorators
- Constants where possible (otherwise use ``...`` instead)
- Necessary import

What can be omitted?

- Private members
- Decorators
- Function implementations (use docstring, ``...`` or `pass`)

Now we have PEP-561, which says that any package maintainer that wants to support stub files has to put a marker file named `py.typed` in the package. We create an empty file called `py.typed`. This is an indicator for the static analytics tool for our IDE that it should look for the stub files in the

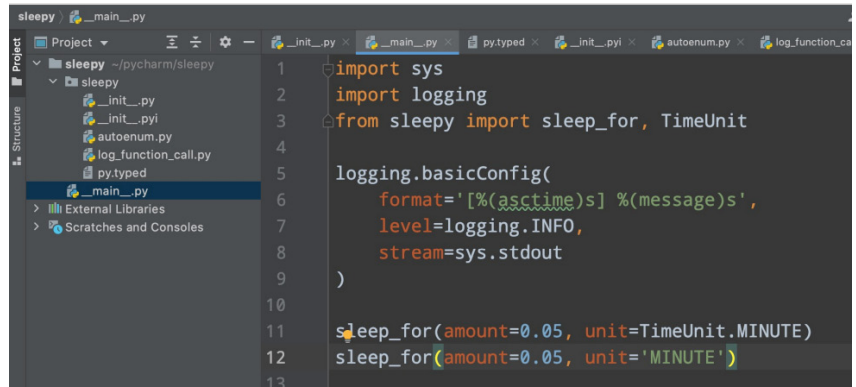


Figure 8: Screen with highlighting error

package. Let's see what happens to our IDE hinting after we add a `py.typed` empty file.

We can once again see the signature and the autocomplete. We typed an argument that is not there and it is still highlighted. If we pass a string instead of the `Timeunit.num`, you can see it is not highlighted as an error in Figure 8. But there is still a problem. The reason for using all those decorators was to eliminate the boilerplate so that our coding process could be simplified and there would be no need to write a new piece of code every time. We could put this logic to decorators, but to support the IDE correctly we need to maintain stub files all by ourselves. This is a huge burden and defeats the purpose of using the decorators in the code. This is where automatic stub generation comes in.

Automatic stub generation

To cut the long story short, the IDE is using static code analysis because it is afraid to run our code and does not have access to runtime objects to examine them. But if we write our code correctly while ensuring that it does not contain any side effects, and we can safely import it, then we can help the IDE to import these modules. From the imported modules we can inspect the module and create stub files without decorators, but the IDE will be able to pick it.

Let us write some code to see how

this can be implemented:

```
import importlib.util
from inspect import isfunction, signature
from io import StringIO
from types import ModuleType
def load_module_from_file (name: str,
path: str):
    spec = importlib.util.spec_from_
file_location (name, path)
    module = importlib.util.module_
from_spec(spec)
    spec.loader.exec_module(module)
    return module
def generate_stub_content (module:
ModuleType) -> str:
sio = StringIO()
    for name, value in module.__dict_.
items():
        if isfunction(value) and
value.__module__ == module.__name__:
            sio.write(f'def {name}{signature
(value)}:\n    pass\n')
    return sio.getvalue()
def main(name, path):
    with open (path + 'i', 'w') as
stub_file:
        module = load_module_from_
file(name, path)
        stub_file.write(generate_stub_
content(module))
if __name__ == '__main__':
    main('sleepy', '../sleepy/__init__.py')
```

First, we have a function that receives the name of a module and a path to a source code. It imports the